Software Architecture

231 terms · name, then the phrase to paste · Workbook Library

Architectural styles

Monolith a single deployable monolith with one database, run as several stateless copies behind a load balancer; keep sessions and files out of local memory and disk so any copy can serve any request

Modular monolith a modular monolith: one deployable split into modules by business capability, each owning its own schema and exposing a small public interface; no cross-module table access or imports of internals, enforced by an architecture test in CI

Microservices microservices split by business capability, one team per service, each with its own datastore and CI/CD pipeline; synchronous calls only at the edge, events between services, and no shared database or shared domain library

Service-oriented architecture a service-oriented integration layer where systems talk through a central bus with canonical message formats; keep the bus to routing and format translation and keep business rules in the services

Serverless functions serverless functions triggered by HTTP, queue and schedule events, one function per use case, stateless with state in a managed database; set timeouts and concurrency limits per function and send failed events to a dead-letter queue

Right-sized services a small number of right-sized services (3 to 6), one per team and business area, each a modular monolith inside with its own database; split further only for a measured scaling, security or release-cadence need

Cell-based architecture a cell-based architecture: identical full-stack cells (app, cache, database) each serving a fixed set of tenants, a thin stateless router mapping tenant to cell, deploys rolled out cell by cell, and no calls between cells

Space-based architecture space-based processing: stateless processing units that read and write a replicated in-memory data grid, with an asynchronous data pump persisting changes to the database and a reader that reloads the grid on cold start

Client-server and n-tier a 3-tier layout: a thin client, a stateless application tier that holds all business rules, and a database reachable only from the application tier, each tier in its own network zone

Peer-to-peer a peer-to-peer design where each node runs the same code, discovers peers through a Kademlia-style DHT with bootstrap nodes, exchanges content-addressed chunks directly and verifies every chunk by hash

Micro-frontends micro-frontends composed by a thin app shell that owns routing, auth and the design system; each team ships its fragment and its backend independently, fragments talk only through URL and custom DOM events, and shared libraries are pinned as singletons

Self-contained systems self-contained systems: each business area is a separate web app with its own UI, logic and database, integrated through hyperlinks and server-side includes, sharing data only through async feeds, never by synchronous calls on the request path

Local-first local-first: the UI reads and writes a local SQLite/IndexedDB store, mutations are queued and synced in the background through a sync server, and concurrent edits merge with CRDTs (Yjs or Automerge) so the app works fully offline

Edge-first an edge-first setup: request handlers run on edge workers near users, serving reads from edge KV or cache with short TTLs, and forwarding writes to a single-region origin database with read-your-writes handled by a session token

Layered architecture a strict layered structure (controller, service, repository): controllers only parse and validate input, services hold the business rules and transactions, repositories hold all queries, and no layer skips the one below it

Hexagonal architecture hexagonal (ports and adapters): the core defines inbound use-case ports and outbound ports such as OrderRepository and PaymentGateway; HTTP, CLI and tests are driving adapters, Postgres and Stripe are driven adapters, and the core imports no framework or SDK

Clean architecture clean architecture with four rings (entities, use cases, interface adapters, frameworks); dependencies point only inward, each use case is one class with an input and output DTO, and frameworks and the ORM live only in the outer ring

Onion architecture onion architecture: the domain model in the center with no dependencies, repository interfaces declared in the domain services layer, application services orchestrating use cases, and infrastructure implementing those interfaces from the outer ring via dependency injection

Vertical slice architecture vertical slices: one folder per use case (e.g. PlaceOrder/) holding its endpoint, request/response types, handler, validation and data access; slices never call each other, and shared code moves into the domain only after the second real use

Package by feature package by feature: top-level folders per business feature (orders/, billing/), each holding its own routes, service and data access, with one index file as the public API; other features import only from that index

Bounded contexts split the domain into bounded contexts, each with its own model and ubiquitous language and one owning team; draw a context map and put an anti-corruption layer wherever a context consumes another context's or a legacy system's model

Aggregates, entities and value objects rich aggregates: each aggregate root guards its invariants in methods (no public setters), uses immutable value objects for money, quantities and ids, references other aggregates by id only, is saved one per transaction and records domain events

Functional core, imperative shell functional core, imperative shell: business decisions as pure functions over plain data that return a list of effects; a thin shell loads inputs, calls the core and executes the effects, and only the shell touches the database, network or clock

Microkernel a microkernel: a minimal core with a plugin registry and versioned extension points (typed hooks), plugins declared by manifest and loaded in isolation, so a failing plugin is disabled without taking down the host

Pipes and filters a pipes-and-filters pipeline of small single-purpose stages with one shared record type, connected by bounded queues or streams with backpressure; each stage is pure where possible and testable on its own

MVC MVC with thin controllers that only map requests to model calls and pick a view, models that hold the business rules, and views with no logic beyond formatting

MVVM MVVM: one view model per screen exposing immutable observable UI state and command methods, no references to view or platform types, and all data access through injected repositories so the view model is unit-tested without UI

Unidirectional data flow unidirectional data flow: a single immutable state tree, views that only render state and dispatch typed actions, a pure reducer (state, action) => state, and side effects isolated in middleware or effect handlers

Actor model the actor model: one actor per entity (e.g. per device or chat room) owning its state, processing its mailbox one message at a time, communicating only by async messages, under supervisors that restart crashed children (let it crash)

Entity component system an entity component system: entities as plain ids, components as plain data in contiguous arrays per type, and systems as functions that query for a set of components and run in a fixed order each tick

Events & messaging

Message queue a point-to-point message queue between the producer and the worker: each message is handled by one consumer, acked only after the work commits, with a visibility timeout longer than the slowest expected handler

Publish/subscribe publish/subscribe for order events: one topic per event type, one durable queue per subscribing service, so each subscriber gets its own copy and fails or lags independently

Event streaming log a Kafka-style partitioned log for domain events, 7-day retention, consumer groups that commit offsets only after processing, and the ability to reset a group's offset to replay

Competing consumers competing consumers on the jobs queue: N stateless workers pulling with a prefetch of 10, autoscaled on queue depth and oldest-message age, with handlers that tolerate parallel and duplicate delivery

Partition keys and ordering partition events by entity id (order_id as the message key) so all events for one order are processed in order by one consumer, with no ordering assumed across different orders

Delivery guarantees at-least-once delivery everywhere (ack only after the handler's transaction commits) plus deduplication by message id in the consumer, giving effectively-once processing

Idempotent consumer idempotent message handlers: state changes as conditional updates or upserts keyed on business ids, and the message id passed as the idempotency key to every external call with side effects

Dead letter queue a dead letter queue per consumer queue: after 5 failed attempts with backoff the message moves to the DLQ with the error attached, an alert fires when DLQ depth is above 0, and there is a one-command redrive

Poison message handling poison message handling: schema or parse failures are parked to the DLQ immediately with the error and raw payload, other failures after a max delivery count of 5, so one bad message never blocks the partition

Backpressure explicit backpressure: bounded in-memory queues (max 1,000 items) that block producers when full, pull-based consumers with a fixed prefetch, and HTTP 429 with Retry-After at the edge when the backlog is over its limit

Request-reply over messaging request-reply over the broker: each request carries a correlation id and a replyTo queue, the requester keeps a map of pending ids with a 5 s timeout, and replies with unknown or expired ids are logged and dropped

Background job queue a background job queue (BullMQ on Redis) for slow work: the request enqueues a job with ids only, never full objects, and returns 202; workers retry 5 times with exponential backoff, and every job is safe to run twice

Scheduled and delayed jobs scheduled and delayed work through the job queue: delayed jobs with a deterministic job id so they can be cancelled, cron schedules in UTC that run once per tick across all workers, and a stated policy for missed runs

Event-driven architecture event-driven integration between services using the broker topology for simple fan-out reactions and a mediator (orchestrator) only where a process has ordered steps and error handling

Event notification thin event notifications: events carry the event type, entity id, version and timestamp only, and consumers fetch current state from the owning service's API when they need it

Event-carried state transfer event-carried state transfer for customer data: CustomerUpdated events carry the full current customer snapshot, published to a compacted topic keyed by customer id, and each consumer keeps its own read-only copy

Domain events vs integration events separate domain events (in-process, internal to the module, free to change) from integration events (versioned public contracts with a documented schema); only integration events leave the bounded context, via the outbox

Event sourcing event sourcing for the Account aggregate only: an append-only event stream per account with optimistic concurrency on the expected version, state rebuilt by folding events, snapshots every 100 events, and read models built by projections

CQRS CQRS for the orders area: commands go through the domain model to the write database, queries read denormalized read models updated from events, and the UI handles read-your-own-writes by returning the new version from commands

Projections and read models projections that build one read table per query from the event stream, store their checkpoint in the same transaction as the read-model write, are idempotent per event, and can be rebuilt from position 0 into a new table then swapped

Saga (choreography) a choreographed saga for checkout: each service reacts to the previous event, commits locally, and emits the next event or a failure event that triggers compensation upstream; a correlation id on every event and a dashboard of in-flight sagas

Saga (orchestration) an orchestrated saga for checkout: a persisted orchestrator that sends reserve, charge and ship commands one at a time, waits for replies with per-step timeouts, and runs compensations in reverse order on failure

Compensating transactions a compensating action for every state-changing saga step (release stock, refund payment, cancel booking), run in reverse order on failure, idempotent and retried until success, with irreversible steps such as notifications placed last

Durable execution durable execution (Temporal) for the onboarding flow: deterministic workflow code that calls activities for all I/O, each activity with a timeout and retry policy, durable timers for waits, and workflow versioning for changes to running instances

Transactional outbox a transactional outbox: the event row is inserted in the same database transaction as the state change, and a relay publishes outbox rows in order (polling every 500 ms or via Debezium) and marks them sent, at-least-once

Inbox pattern an inbox table (message_id primary key, received_at) written in the same transaction as the handler's changes; a unique violation means a duplicate, so skip and ack; prune rows older than 14 days

Change data capture log-based change data capture with Debezium from the Postgres WAL into Kafka, one topic per table, publishing the outbox table rather than raw business tables to other teams, with alerts on replication slot lag

Event schema registry and versioning versioned event schemas in Protobuf with a schema registry enforcing backward compatibility in CI (add optional fields only, never rename or reuse field numbers), breaking changes as a new event type, and upcasters for old stored events

Data

Database per service database per service: each service owns its tables and is the only writer and reader; other services use its API or its events, never its database, enforced with separate database users or schemas

Shared database one shared Postgres database for the monolith, with a schema per module (orders, billing, catalog); modules write only their own schema and read others only through module APIs or published views

Polyglot persistence Postgres as the source of truth for transactional data; add Redis for sessions and rate limits, OpenSearch for full-text search and S3 for files only where a concrete query needs it, each fed from Postgres

Read replicas one primary for writes and async read replicas for read-only queries; route reads that follow a write in the same session to the primary, and alert when replica lag exceeds 5 s

Sharding shard by tenant_id using hash sharding (Citus or Vitess) so every query for one tenant hits one shard; no cross-shard transactions, and fan-out queries only for admin reporting

Table partitioning Postgres range partitioning by month on created_at for the events table, partitions created ahead by pg_partman, retention by dropping partitions older than 12 months

Multi-tenancy models pool multi-tenancy: every tenant table has tenant_id NOT NULL, Postgres row-level security policies filter on current_setting('app.tenant_id') set per transaction, and indexes lead with tenant_id

Materialized views a Postgres materialized view for the daily sales aggregates with a unique index, refreshed CONCURRENTLY every 5 minutes by a scheduled job, and the page shows the last refresh time

Search index from the primary store Postgres stays the source of truth; changes stream to OpenSearch via Debezium CDC into an idempotent indexer that upserts by id, with a zero-downtime reindex through an index alias

Object storage with metadata in a database files in S3 with metadata in Postgres: the API creates a pending row and returns a presigned PUT URL (5 min expiry, content-type and max size fixed), an S3 event marks the row ready, and a daily job deletes orphans

Time-series store TimescaleDB hypertable for sensor readings partitioned by time, continuous aggregates at 1 minute and 1 hour, raw data retained 30 days, 1-minute rollups 1 year, compression after 7 days

Vector store for retrieval pgvector in the main Postgres: chunks of about 500 tokens with doc_id and an embedding column, an HNSW index with cosine distance, hybrid retrieval with full-text search, top 5 chunks filtered by tenant

OLTP and OLAP split keep analytics off the app database: load Postgres into the warehouse hourly with Airbyte, model it in dbt (staging, then marts) with tests on keys, and point dashboards only at marts

Data lakehouse a lakehouse on S3 with Apache Iceberg tables in a REST catalog: bronze as raw append-only ingests, silver deduplicated and typed, gold aggregated per use case; Spark for transforms, Trino for queries

Repository a repository per aggregate (OrderRepository with get, save and a few named queries), domain code depends on the interface, one Postgres implementation and an in-memory fake for tests; no generic Repository<T>

Unit of work one unit of work per request or command: load entities, change them in memory, then commit once in a single transaction that also writes outbox rows; no commits inside domain methods

Active record active record models (Rails style) with validations and associations on the model, eager loading with includes to avoid N+1, and logic that spans several models moved into service objects

ORM vs query builder vs raw SQL use Kysely as a typed query builder for most data access, raw parameterized SQL for reports, no ORM change tracking; every query visible in code and covered by an integration test against real Postgres

DTOs and mapping at boundaries separate request DTOs (validated with zod), domain types and database rows, with explicit mapping functions at each boundary; never return ORM entities or rows directly from the API

Optimistic concurrency optimistic concurrency with an integer version column: UPDATE ... WHERE id = $1 AND version = $2, zero rows returns 409 Conflict; the API exposes it as an ETag and requires If-Match on PUT and PATCH

Pessimistic locking pessimistic locking with SELECT ... FOR UPDATE inside short transactions, locks taken in a consistent order to avoid deadlocks, lock_timeout of 2 s, and FOR UPDATE SKIP LOCKED for the job queue

Soft delete soft delete with a nullable deleted_at timestamptz, a default scope or view that hides deleted rows, partial unique indexes WHERE deleted_at IS NULL, and a job that hard-deletes after 30 days

Audit log and history tables an append-only audit_log table (actor, action, entity, before and after as jsonb, request id, timestamp) written in the same transaction as the change, with no updates or deletes allowed on it

Expand and contract migrations expand and contract for schema changes: add nullable columns, dual-write and backfill in batches, switch reads, drop the old column in a later release; no locking ALTERs on large tables and indexes built CONCURRENTLY

Cache-aside cache-aside in Redis for product reads: get, on miss load from Postgres and set with a 5 min TTL; writes update Postgres then delete the key; cache errors fall through to the database

Write-through and write-behind caching write-behind for view counters: INCR in Redis on each hit, a worker flushes deltas to Postgres every 10 s in one batched upsert; losing up to 10 s of counts on a crash is acceptable

Cache invalidation and stampede protection stampede protection: TTL 300 s with ±10% jitter, per-key request coalescing (singleflight) so one miss recomputes, serve stale for up to 60 s while refreshing, invalidate by deleting keys on write

Consistency models read-your-writes for the user's own data: after a write, return the commit LSN in a cookie and serve that user's reads from the primary or from a replica that has replayed past it; everything else may read eventually consistent replicas

Idempotency keys for writes idempotency keys on all POSTs: Idempotency-Key header required, stored with a request hash and the response in the same transaction under a unique constraint, replay on repeat, 409 if in flight, 422 if the body differs, keys expire after 24 h

UUIDv7 and ULID keys UUIDv7 primary keys generated in the app (or Postgres 18 uuidv7()), stored as the native uuid type, never random UUIDv4 for clustered or indexed keys

APIs & integration

REST resource API a resource-oriented REST API: plural nouns (/orders, /orders/{id}), standard methods and status codes (201 + Location on create, 204 on delete, 404/409/422 for errors), PATCH with JSON Merge Patch, and Idempotency-Key on POST

GraphQL a GraphQL API with a schema-first SDL, DataLoader batching per request, query depth and cost limits, persisted queries in production, and errors in the errors array with stable extension codes

gRPC gRPC between internal services: .proto contracts in a shared repo linted and breaking-change checked with Buf, deadlines set on every call, status codes mapped deliberately (UNAVAILABLE retryable, INVALID_ARGUMENT not)

Typed RPC typed RPC with tRPC: one router per feature, Zod input validation on every procedure, queries vs mutations kept distinct, and the AppRouter type imported by the client so a server change breaks the client build

JSON-RPC JSON-RPC 2.0: namespaced methods (tools/call, orders/get), every request carries an id, standard error codes (-32600, -32601, -32602) plus app codes from -32000, and batch requests supported

Webhooks webhooks per the Standard Webhooks spec: HMAC-SHA256 signature over id + timestamp + body, 5-minute timestamp tolerance, retries with exponential backoff for up to 3 days, and receivers that return 2xx fast, enqueue, and dedupe by event id

WebSockets WebSockets with auth on the upgrade request, ping/pong heartbeats every 30 s, client reconnect with jittered backoff and resume from last seen sequence number, and fan-out across nodes via Redis pub/sub

Server-sent events server-sent events: text/event-stream with an id on every event, resume from Last-Event-ID, a comment heartbeat every 15 s to keep proxies from closing the stream, and proxy buffering turned off for that route

Long polling long polling with a 25 s server hold (below every proxy timeout), a since cursor on each request, 200 with events or 204 on timeout, and immediate re-poll on the client

Async request-reply async request-reply: POST returns 202 Accepted with Location: /jobs/{id} and Retry-After; the job resource reports queued, running, succeeded or failed with progress, and on success links to the result; jobs are kept 7 days

Contract-first API contract-first with OpenAPI 3.1 in the repo: lint with Spectral, generate typed clients and server request validation from it, run oasdiff in CI to block breaking changes, and validate real responses against the spec in tests

API versioning API versioning by additive evolution first (new optional fields and endpoints only, never rename, retype or remove), a major version in the URL (/v2) only for real breaks, and Deprecation + Sunset headers with usage tracking before removing anything

Cursor pagination cursor pagination: opaque base64 cursor encoding (created_at, id), WHERE (created_at, id) < cursor ORDER BY created_at DESC, id DESC, limit default 50 max 200, response returns next_cursor and has_more, backed by a composite index

Rate limiting and quotas rate limiting per API key with a token bucket (100 req/s, burst 200) in Redis at the gateway, 429 with Retry-After and RateLimit headers when exceeded, separate stricter limits on login and expensive endpoints, and clients that honor Retry-After

Problem details errors errors as RFC 9457 problem details (application/problem+json) with a stable type URI per error kind, a field-level errors array for 422 validation failures, a traceId extension, and no stack traces or SQL in detail

Consumer-driven contract tests consumer-driven contract tests with Pact: consumers publish pacts from unit tests to a Pact Broker tagged by branch and environment, providers verify them in CI, and every deploy is gated by can-i-deploy

API gateway an API gateway that terminates TLS, validates JWTs, applies per-key rate limits, routes by path to services, and adds a request id; no business logic or response reshaping in the gateway

Backend for frontend a backend for frontend per client (web, mobile), owned by that client team, that aggregates downstream calls, shapes responses per screen, keeps OAuth tokens server-side behind an httpOnly session cookie, and holds no domain rules

Service mesh a service mesh (Linkerd or Istio ambient) providing automatic mTLS between services, per-route timeouts and retry budgets set in mesh config rather than app code, and golden-signal metrics per route

Sidecar a sidecar container (Kubernetes native sidecar) next to the app that ships logs from the shared volume and fetches secrets into it, with its own CPU and memory limits, started before and stopped after the app

Ambassador an ambassador proxy on localhost that the app calls for the payments provider, owning TLS, connection pooling, 2 s timeouts, retries with jittered backoff on 503 only, and a circuit breaker; the app itself does no retries

Anti-corruption layer an anti-corruption layer between our domain and the legacy ERP: one module that maps ERP records to our domain types (and back), validates them, and is the only code that imports ERP client types

Strangler fig migration a strangler fig migration: a proxy in front of the monolith routes migrated paths to the new service and everything else to the monolith, with each slice switchable per route via config and shadow traffic compared before cutover

Facade over legacy a facade service exposing a small REST API (GET /customers/{id}) over the mainframe, CRM and billing database, with its own response model, per-backend timeouts, and short-lived caching of slow lookups

Adapter at the boundary an adapter per external service: an interface in our domain terms (Payments.charge), one adapter class that alone imports the vendor SDK and maps its types and errors to ours, and an in-memory fake for tests

Reverse proxy and load balancer an L7 load balancer (Envoy or ALB) that terminates TLS, routes by host and path, balances per request with least-request, health-checks backends with outlier ejection, and drains connections on deploy

Service discovery service discovery through Kubernetes Services and DNS names (orders.default.svc), readiness probes that gate registration, and callers that never hard-code instance IPs

Edge functions and CDN a CDN in front of the API caching public GET responses with Cache-Control s-maxage and stale-while-revalidate, never caching responses that vary by user, and an edge function that verifies the JWT and rejects bad requests before the origin

File and batch integration batch file integration: nightly Parquet export to S3 under dated prefixes with a _SUCCESS manifest holding row count and schema version, an idempotent loader that upserts by key, and an alert if the file is late or counts do not match

API composition API composition: fan out to orders, profile and reviews in parallel with per-call timeouts (300 ms), mark reviews optional so a timeout returns the page without them, and an overall deadline of 800 ms

Reliability & scale

Timeouts and deadlines explicit timeouts on every outbound call (connect 1 s, request set from measured p99) and a per-request deadline propagated downstream (gRPC deadline or a header with time remaining); work stops when the deadline passes

Retries with backoff and jitter retries with exponential backoff and full jitter (base 100 ms, cap 10 s, max 4 attempts), only for idempotent operations and only on timeouts, connection errors, 429 and 503; honor Retry-After when present

Retry budget a retry budget per client: retries limited to 10% of successful request volume (plus a small floor of 10/s), retry only at one layer of the stack, and no retry when the budget is spent

Circuit breaker a circuit breaker per downstream dependency: opens when the failure rate passes 50% over a rolling window of at least 20 calls, fails fast for 30 s, then half-opens and allows 3 probe calls before closing

Bulkhead bulkheads: a separate bounded connection pool and concurrency limit per downstream dependency (e.g. 20 for payments, 50 for search), so a slow dependency can exhaust only its own pool

Fallback and graceful degradation graceful degradation: each non-critical dependency (recommendations, reviews) has a defined fallback (cached last-good value or hidden section) used on timeout or open breaker; core flows like checkout never depend on them

Load shedding load shedding at the edge of each service: an adaptive concurrency limit, and when it is reached reject low-priority requests first with 503 and Retry-After, keeping checkout and auth paths admitted

Queue-based load leveling queue-based load leveling: producers enqueue jobs to a durable queue, a fixed-size worker pool pulls at a steady rate, with alerts on queue age (oldest message > 5 min) and a dead-letter queue after 5 failed attempts

Rate limiting rate limiting with a token bucket per API key (refill 10/s, burst 20) stored in Redis, returning 429 with Retry-After and RateLimit headers; stricter per-IP limits on login and signup

Health checks separate health endpoints: /livez checks only the process itself (no dependencies), /readyz checks it can serve (warm, connected to its own DB) and fails during shutdown drain, and a startup probe for slow boots

Hedged requests hedged reads: if a replica has not answered within the p95 latency, send one backup request to another replica, take the first response and cancel the other; reads only, and cap hedging at 5% of traffic

Idempotent operations idempotent writes: clients send an Idempotency-Key header, the server inserts the key under a unique constraint before doing the work, stores the response for 24 h and replays it on retries with the same key

Chaos engineering and game days chaos experiments: a written steady-state hypothesis (e.g. p99 < 300 ms, errors < 0.5%), faults injected in staging first and then in production with a small blast radius and an abort switch, run on a schedule

Stateless services, horizontal scaling stateless app instances: no in-memory sessions or local files, sessions in Redis or signed cookies, uploads straight to object storage, so any instance can serve any request behind the load balancer

Vertical scaling scale up before scaling out: measure the bottleneck, move the database to a larger instance with a failover replica, and only split or shard once the largest sensible instance is not enough

Autoscaling autoscaling with explicit bounds (min 2, max 20): API pods target 60% CPU, worker pods scale on queue depth (about 100 messages per pod) via KEDA, with a 5 min scale-down cooldown

Caching layers caching in layers: Cache-Control with max-age and stale-while-revalidate for the browser and CDN, a read-through Redis cache with TTLs and explicit invalidation on writes, and request coalescing on misses to avoid stampedes

Consistent hashing consistent hashing with virtual nodes (about 100 per physical node) to map keys to cache or shard nodes, so adding a node moves only about 1/N of the keys

Leader election leader election through a lease in etcd or a Kubernetes Lease (TTL 15 s, renew every 5 s); only the leader runs the scheduler, and its writes carry a fencing token that storage checks

Consensus use an existing consensus system (etcd, ZooKeeper or Consul) for coordination state, deployed as 3 or 5 nodes across zones; do not implement a consensus protocol ourselves

Multi-region active-passive active-passive multi-region: primary region serves all traffic, async database replication to a warm standby region, DNS failover with health checks, target RPO under 1 min and RTO under 30 min, failover drilled each quarter

Multi-region active-active active-active across two regions with geo-routed DNS; each record has a home region for writes (or uses a database with multi-region consensus like Spanner or CockroachDB) so we avoid last-writer-wins conflicts

CAP and PACELC in practice state the consistency choice per data type: balances and inventory use strongly consistent (linearizable) reads and writes, while feeds, counters and search tolerate eventual consistency with bounded staleness

Blue-green deploys blue-green deploys: deploy to the idle environment, run smoke tests against it, flip the load balancer target, keep the old environment for 1 h for instant rollback; schema changes follow expand-and-contract

Canary releases canary releases: 1% → 10% → 50% → 100% with 15 min per step, automatic rollback if the canary's 5xx rate or p99 latency is worse than the stable version by a set margin

Rolling deploys rolling deploys with maxSurge 25% and maxUnavailable 0, readiness probes gating each step, graceful shutdown that drains connections for 30 s, and old and new versions compatible with each other

Feature flags and dark launches feature flags through OpenFeature with percentage and per-account targeting, a kill switch for each risky feature, every flag with an owner and removal date; new backends dark-launched with shadow traffic first

Structured logging structured JSON logs with consistent fields (ts, level, service, msg, trace_id, request_id, user_id when known), event names as stable message keys, and no secrets or full request bodies

Metrics with RED and USE metrics by the RED method for every endpoint (request rate, error rate, latency histogram with p50/p95/p99) and USE for each resource (CPU, memory, connection pools, queue depth), exported via OpenTelemetry or Prometheus with low-cardinality labels

Distributed tracing distributed tracing with OpenTelemetry SDKs in every service, W3C traceparent propagated over HTTP and message headers, spans around each external call, and tail-based sampling that keeps all errors and slow traces

Correlation and request ids a request id generated at the edge (or taken from an incoming X-Request-Id), carried in async context, added to every log line, forwarded on outbound calls and queue messages, and returned in error responses

SLIs, SLOs and error budgets SLOs per user-facing journey (e.g. 99.9% of checkout requests succeed and 99% finish under 500 ms, over 30 days), with an error budget policy that pauses risky releases when the budget is spent

Alert on symptoms, not causes alerts that page only on user-facing symptoms: multi-window SLO burn rate (e.g. 14x over 1 h and 5 min) and error rate on key endpoints; resource alerts like CPU or disk create tickets, not pages

Dashboards and runbooks every paging alert includes links to a runbook (meaning, how to confirm, first mitigation such as rollback, escalation) and a service dashboard showing RED metrics, deploy markers and dependency health

Audit trail vs debug logs a separate audit trail table (actor, action, target, before/after, ip, timestamp) written in the same transaction as the change, append-only and retained for 7 years; debug logs stay separate, sampled and kept 14 days

Design patterns

Factory function a factory function (createX(config): X) that picks the concrete implementation from config and returns it behind an interface; callers never call constructors of concrete classes directly

Abstract factory an abstract factory interface that creates a matching family of objects (one method per product), chosen once at startup and passed in, so families can never be mixed

Builder a builder for objects with more than ~4 optional fields: chained setters, a build() that validates required fields and returns an immutable value; use a typed options object with defaults where that is enough

Dependency injection constructor injection with plain constructors and no DI container: every dependency (repos, clock, HTTP clients, config) is passed in, and everything is created and wired once in a single composition root (main.ts)

Singleton shared resources (db pool, HTTP client, logger, config) created exactly once in the composition root at startup and passed to whoever needs them; no getInstance() and no module-level instances that read env on import

Object pool a bounded pool for expensive resources (max size set from the downstream limit, idle timeout, acquire timeout); always release in finally or a using/defer block, and expose pool wait time as a metric

Adapter wrap each third-party SDK in an adapter that implements an interface we own; translate vendor types and errors into our domain types at the adapter, and never import the SDK outside it

Decorator cross-cutting concerns (caching, logging, metrics, retries) as decorators that implement the same interface as the wrapped object, composed in the composition root in an explicit order

Facade a facade function per use case (checkout, refund) that orchestrates the subsystems in order; it holds no business rules itself and the subsystems stay independently testable

Proxy a proxy that implements the same interface as the real object to add lazy initialization or permission checks; remote proxies must expose timeouts and errors in their signatures

Composite a composite tree where leaves and containers implement one interface (size(), render()), containers delegate to their children recursively, and callers never check which kind they hold

Bridge a bridge: separate the abstraction (notification type) from the implementation (delivery channel) behind an interface, and compose them at runtime instead of subclassing each combination

Flyweight flyweights for repeated immutable data: a cache keyed by value that returns one shared frozen instance, with per-use state (position, style) passed in as arguments

Strategy the strategy pattern as a function type (e.g. type Backoff = (attempt: number) => number) passed in by the caller, so variants are added without editing the code that uses them

Observer an in-process typed event emitter (event name to payload type map); listeners run isolated so one failing listener is logged and does not break the others, and every subscribe returns an unsubscribe function

Command user actions as command objects with run() and undo() that capture the state they need; an undo stack and a redo stack, cleared on any new command

State machine model the lifecycle as an explicit state machine: a union of named states, a transition table of (state, event) to next state, and invalid transitions rejected; use XState if it needs nesting, parallel states or timers

Chain of responsibility a middleware pipeline where each step is (ctx, next) => Promise, ordered explicitly in one place: request id, logging, auth, rate limit, body parsing, then the handler; steps may return early with a proper status

Template method fixed workflows as a plain function that takes its varying steps as function parameters, rather than an abstract base class with overridable hook methods

Visitor operations over the AST as visitors (one method per node type) or, in TypeScript, a tagged union with an exhaustive switch and a never check so adding a node type fails to compile until every operation handles it

Iterator and generator paginated or streamed data exposed as an async generator so callers use for await and can break early; no loading everything into memory

Mediator a mediator that owns the interaction rules between sibling components; components emit events to it and receive updates from it, and never reference each other directly

Specification business rules as named, composable predicates (Spec<T> = (x: T) => boolean with and/or/not helpers) defined once in the domain module and reused for validation and filtering

Memento undo via snapshots: the object exposes save() returning an immutable snapshot and restore(snapshot), and history keeps a capped list of them (e.g. last 100)

Interpreter a small DSL as a typed AST (tagged union of node types), a parser that returns errors with positions, and a recursive evaluate function; no eval() and no string concatenation into SQL

Result type expected failures returned as a Result (a tagged union of { ok: true, value } | { ok: false, error }) with a typed union of error codes; throw only for programmer errors and truly unexpected failures

Option instead of null strict null checks on; lookups return T | undefined (or Option<T>) and callers handle the missing case explicitly; optional collaborators like logger default to a no-op null object instead of null checks

Parse, don't validate parse, don't validate: every external input (request bodies, env, messages, third-party responses) goes through a schema (Zod/Pydantic) at the boundary that returns a typed value or a 400 with field errors; inner code takes the parsed type and never re-validates

Make illegal states unrepresentable make illegal states unrepresentable: model modes as a discriminated union on a status field, each variant carrying only its own fields, and handle them with exhaustive switches that end in a never check

Immutability by default immutability by default: readonly types and const, updates return new objects via spread or Immer, no mutating function arguments; local mutation only inside a function where it never escapes

Pure core, effects at the edges functional core, imperative shell: business rules as pure functions that take all inputs (including now and ids) as arguments and return decisions; I/O (db, http, clock, random) only in a thin outer layer that executes those decisions

Typestate and branded types encode states and ids in types: typestate (generic state parameter, transitions consume self) for multi-step protocols, and branded/newtype ids (UserId, OrderId) created only by a parser so they cannot be mixed up

Pipeline and function composition data transformations as a pipe of small named pure functions, one concern each, readable top to bottom; no clever point-free chains

Plugin registry a plugin registry: a typed plugin interface, register() at startup, lookup by id with a clear error for unknown ids; core code depends only on the interface and never imports a plugin

Validated config schema config read only in config.ts: process.env parsed with a Zod (or Pydantic Settings) schema at startup with defaults and coercion, crashing with a list of bad keys on failure; everything else imports the typed config object, never process.env

Feature folders with a public API feature folders (src/features/<name>) each exporting a small explicit public API from index.ts; other features import only from that index, deep imports are blocked by a lint rule, and barrels list named exports (no export *)

Principles & pitfalls

Single responsibility single responsibility: each module or class has one reason to change (one actor), so pricing rules, PDF rendering and persistence live in separate units

Open/closed open/closed along the axis that actually varies: new variants are added as new implementations registered in one map, without editing existing branches

Liskov substitution Liskov substitution: every implementation honors the full interface contract (no "not supported" throws, no stricter preconditions); if a variant cannot, split the interface

Interface segregation interface segregation: each caller depends on a small role interface with only the methods it uses; one class may implement several roles

Dependency inversion dependency inversion: the domain defines the interfaces it needs (e.g. PaymentGateway), adapters in infrastructure implement them, and wiring happens in one composition root

DRY and the rule of three DRY for business rules, rule of three for code: allow a second copy, extract on the third, and keep look-alike code separate when it changes for different reasons

YAGNI YAGNI: implement only what this task needs; no speculative options, plugin points, config flags or generic layers for requirements nobody has asked for

KISS keep it simple: the fewest moving parts and systems that meet the requirement, plain functions over frameworks, and no new infrastructure without a measured need

Separation of concerns separation of concerns: HTTP handlers only parse, authorize and map status codes; business rules live in plain domain functions with no I/O; SQL stays in the repository layer

High cohesion, low coupling high cohesion and low coupling: group code that changes together into one module, and let modules talk only through a narrow public interface with no shared internals

Law of Demeter law of Demeter: no chains through other objects' internals (order.customer.address.city); ask the direct collaborator for what you need and let it delegate

Composition over inheritance composition over inheritance: pass collaborators in (strategies, policies, small functions) instead of extending base classes; inheritance at most one level, for true is-a relationships

Conway's law Conway-aware boundaries: one owning team per service or module, boundaries drawn where team boundaries are (or should be), and no component that needs two teams to change

Choose boring technology choose boring technology: build on the stack we already run (Postgres, Redis, our main language and framework); any new datastore, language or infrastructure needs a written reason and spends one of a few innovation tokens

The twelve-factor app twelve-factor: config from environment variables, stateless processes that can be killed any time, backing services addressed by URL, logs as JSON to stdout, and the same build artifact promoted across environments

Design for failure design for failure: every outbound call has a timeout and a defined fallback (cached value, default, or a clear degraded response), and non-critical features degrade instead of failing the request

Principle of least privilege least privilege: each service gets its own credentials scoped to the exact resources and actions it uses (read-only where it only reads), with no shared admin keys and short-lived tokens where possible

Deep modules deep modules: small, simple public interfaces that hide substantial complexity (defaults, retries, edge cases) inside, and no shallow pass-through wrappers

Architecture decision records record the decision as an ADR in docs/adr/NNNN-title.md with Status, Context, Decision and Consequences (including the downsides and when to revisit)

One-way and two-way doors treat reversible choices as two-way doors (pick a sensible default and move on, behind a flag or interface if cheap) and flag one-way doors (public API shape, schema, vendor) for explicit review

Fitness functions fitness functions in CI: dependency rules (dependency-cruiser, ArchUnit or import-linter) that fail the build on forbidden imports between modules and on cycles, plus a budget check for the one quality attribute that matters most

C4 model diagrams describe the design with C4: a context diagram (users and external systems) and a container diagram (each deployable and datastore, with protocols on every arrow), as Mermaid or Structurizr DSL kept in the repo

Quality attributes and tradeoffs before choosing, list the top 3 quality attributes with concrete targets (e.g. p99 under 200 ms, 99.9% availability, under $500/month), score each option against them and state what we give up

Last responsible moment defer this choice to the last responsible moment: build behind a small interface with the simplest implementation now, and note what we would need to learn before committing

Walking skeleton start with a walking skeleton: one real request flowing through UI, API and database, deployed by the real CI pipeline to a real environment, before building out features

Spike, then throw it away treat this as a timeboxed spike: answer the question in throwaway code on a branch, write down what we learned, and do not merge it

Big ball of mud Avoid a big ball of mud: modules importing each other's internals freely, a catch-all utils package, and no clear owner for data or behavior

Distributed monolith Avoid a distributed monolith: services that must deploy together, share a business-logic library or database, or call each other in long synchronous chains

God object Avoid a god object: one Manager, Helper or Service class that owns many unrelated responsibilities and is imported by most of the codebase

Anemic domain model Avoid an anemic domain model: entities that are only public fields while all rules live in service classes, so any caller can set status or totals directly

Shared database integration Avoid shared database integration: more than one service reading or writing the same tables directly instead of going through the owning service's API or events

Chatty services Avoid chatty service calls: one network or database round trip per item inside a loop, where a batch call, join or DataLoader-style batcher would do

Dual writes Avoid dual writes: saving to the database and publishing to a broker as two separate steps with no outbox, so a crash between them loses or invents events

Golden hammer Avoid a golden hammer: forcing one familiar tool (Kafka, Redis, a framework) onto problems it fits poorly instead of choosing per requirement

Premature abstraction Avoid premature abstraction: interfaces, factories, base classes or plugin points with only one implementation, added for flexibility nobody has asked for

Premature optimization Avoid premature optimization: caches, custom data structures or micro-optimizations added without a profile or benchmark showing they are needed

Leaky abstraction Avoid leaky abstractions: interfaces that pass vendor exceptions, error codes or types (Postgres 23505, AWS SDK errors) through to their callers

Cyclic dependencies Avoid cyclic dependencies between modules or packages, including indirect cycles through a third module

Primitive obsession Avoid primitive obsession: raw strings and numbers for IDs, money, emails and statuses where branded or value types would catch mixups at compile time

Temporal coupling Avoid temporal coupling: objects that need setup methods (init, connect, configure) called in a specific order before they work, with nothing in the types enforcing it

Resume-driven development Avoid resume-driven architecture: microservices, Kubernetes, service meshes or streaming platforms for a small team and modest load that one well-structured app would serve

Magic and hidden global state Avoid hidden global state and magic: mutable globals, singletons, monkey patching, import-time side effects or name-based auto-wiring that make dependencies invisible at the call site